Laboratorio CI/CD – GitHub Actions con Flask y Pytest

Universidad Rafael Landívar · Facultad de Ingeniería · Virtualización – Fase IV
Estudiante: Eduardo David Hernández Girón · **Carné:** 1156823 · 5 de noviembre de 2026
Repositorio: github.com/EduardoHG017/laboratorio-devops

Se creó una aplicación Flask con dos endpoints (`/` y `/health`) y dos pruebas automáticas con Pytest. También se configuró un workflow de GitHub Actions que corre esas pruebas en cada push. El workflow pasó en la primera ejecución, detectó el error que se introdujo a propósito y volvió a pasar después de la corrección.

Herramienta	Versión
Python (local / runner)	3.11.9 / 3.12
Flask	3.1.3
Pytest	9.2.0
Runner	ubuntu-latest

Parte A. GitHub Actions

En la carpeta `laboratorio-devops` se crearon `app.py` y `requirements.txt` (Flask y Pytest), y las dependencias se instalaron en un entorno virtual `.venv`. Al correr la app localmente, `/` respondió `{"mensaje":"Virtualizacion URL"}` y `/health` respondió `{"status":"ok"}`, los dos con código 200.

En `test_app.py` se escribieron dos pruebas, `test_inicio` y `test_health`. Usan el `test_client()` de Flask y validan el código de estado y el JSON de respuesta. Localmente pasaron las dos (`2 passed`).

El workflow `.github/workflows/ci.yml` ("CI Flask") se ejecuta con `push`, `pull_request` y `workflow_dispatch`. Corre en `ubuntu-latest` y hace cuatro pasos: descargar el repositorio, configurar Python 3.12, instalar dependencias y ejecutar `pytest -v`. El `.gitignore` deja fuera `.venv/`, las cachés y los archivos `.env`.



Figura 1. Creación de `app.py`, `requirements.txt` y el entorno virtual.

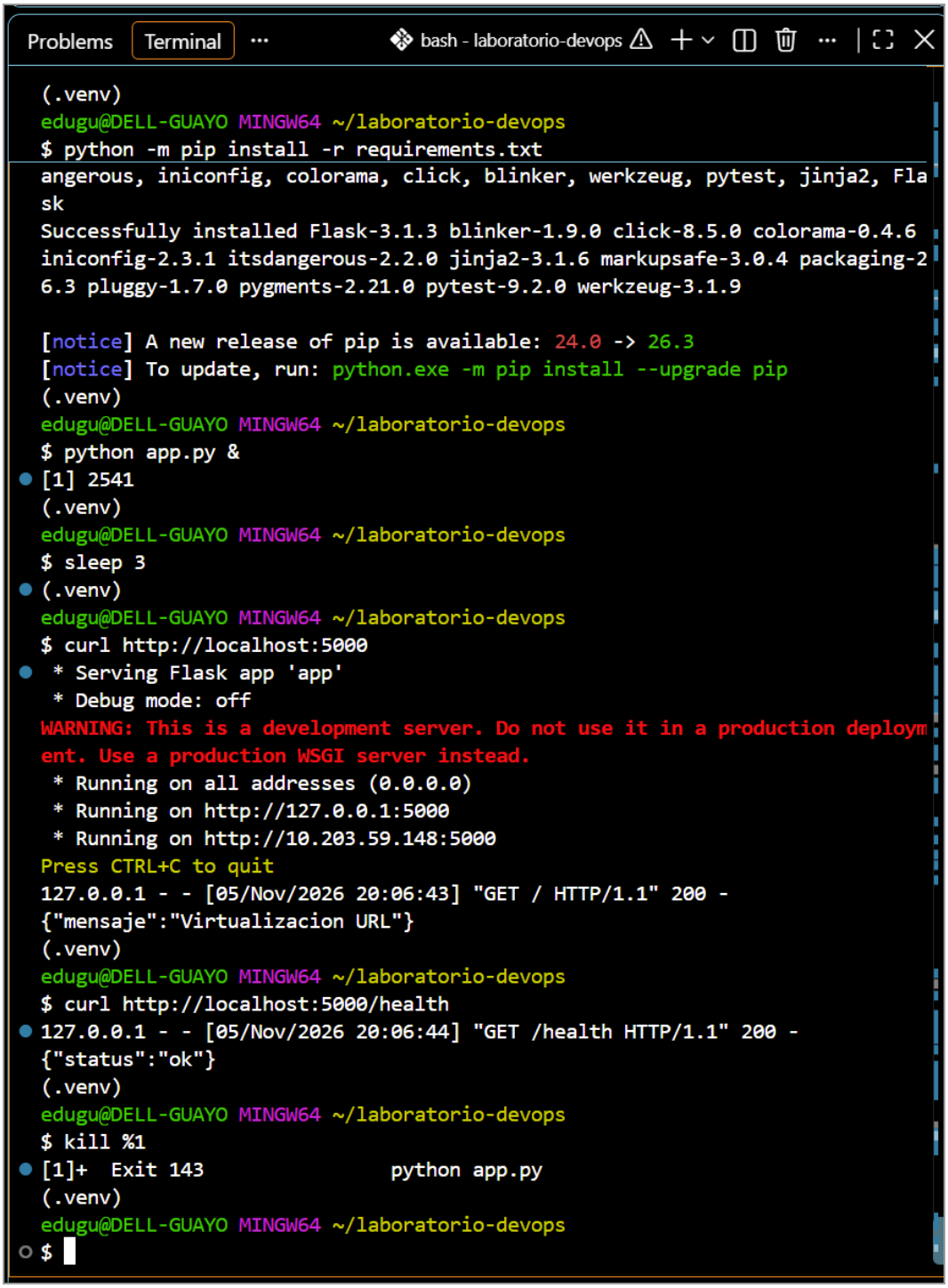


Figura 2. La app corriendo localmente y respondiendo en `/` y `/health`.

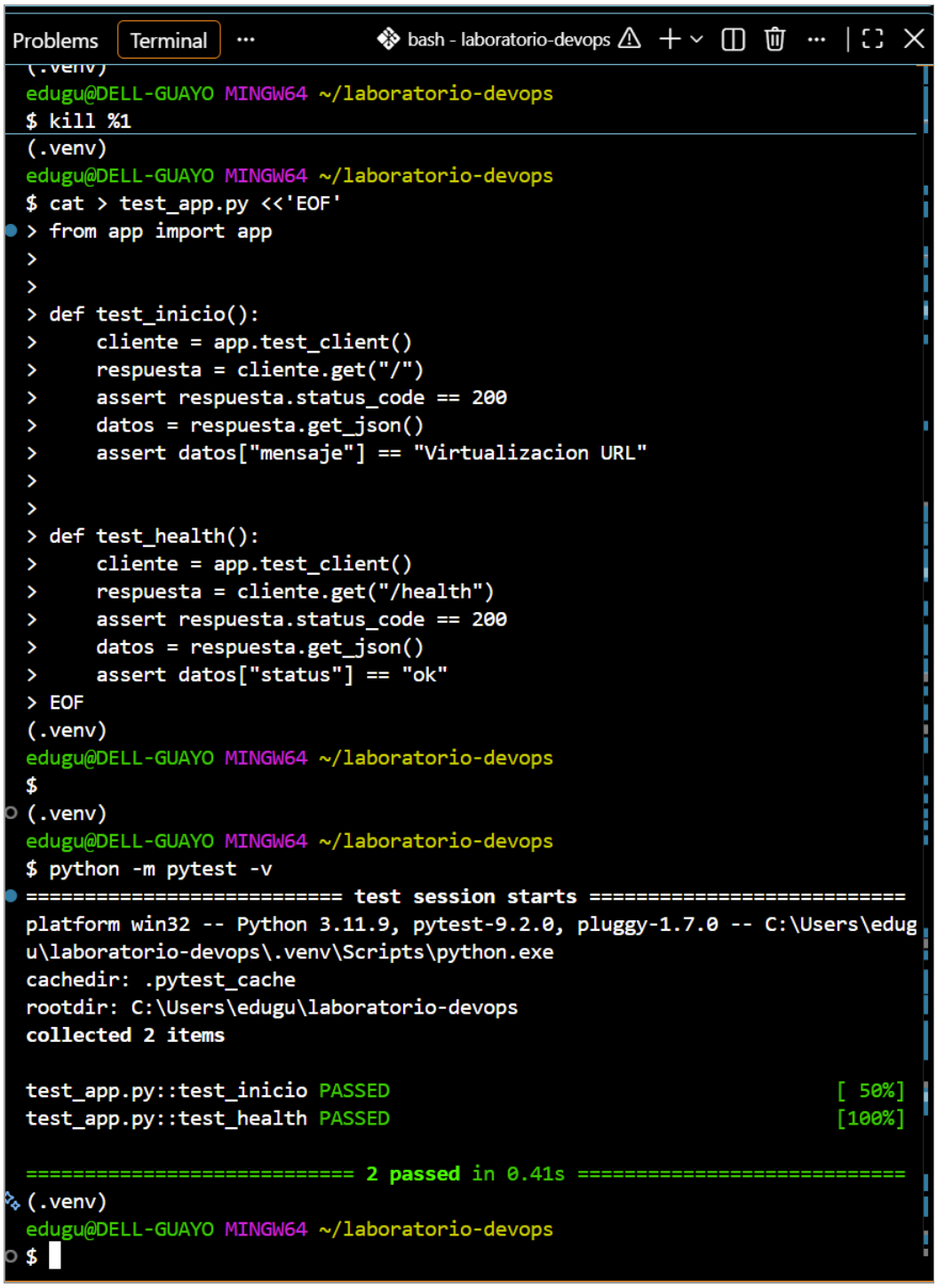


Figura 3. Pruebas locales con Pytest: 2 passed.

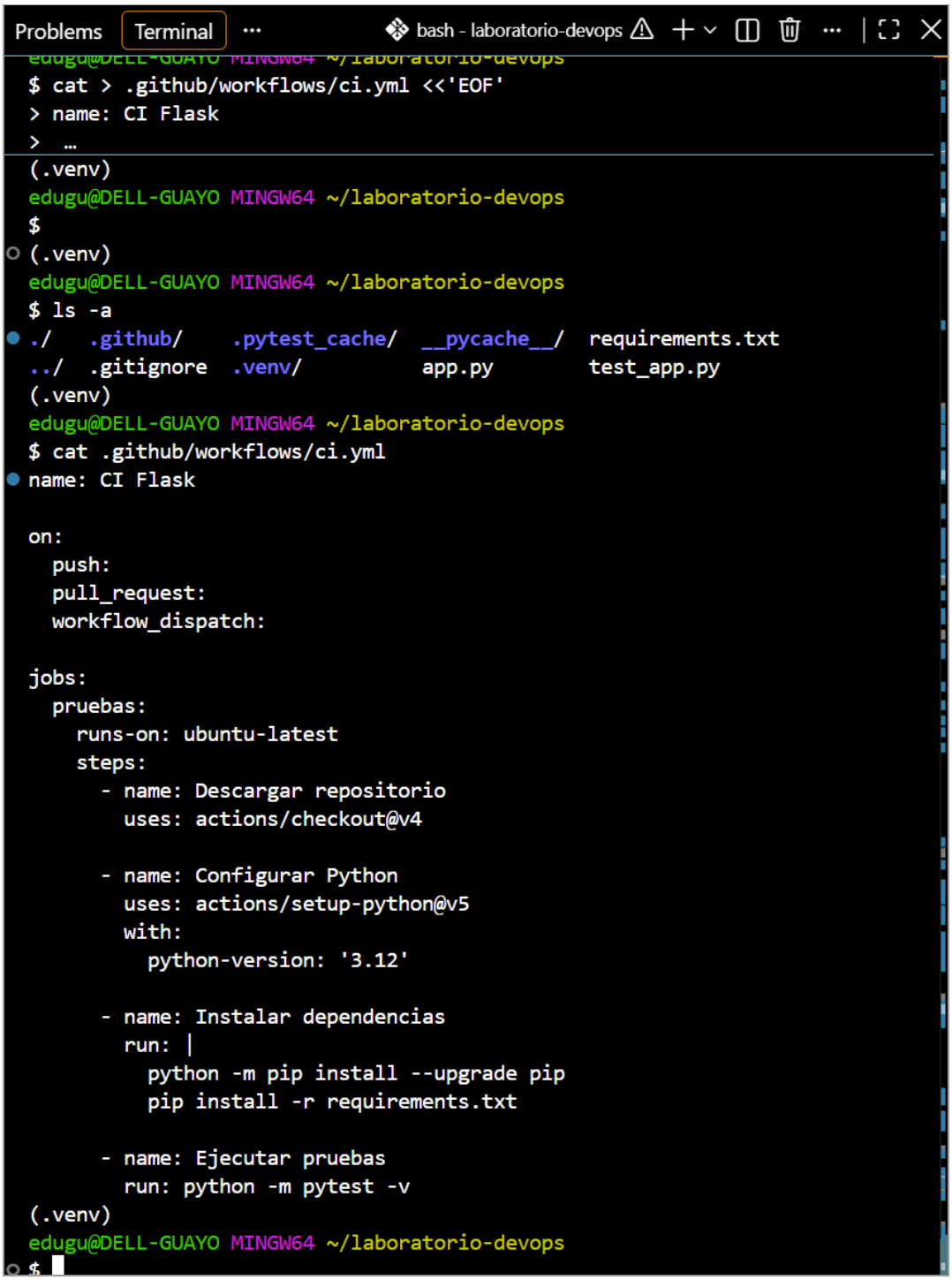


Figura 4. Archivo `ci.yml` y estructura del proyecto.

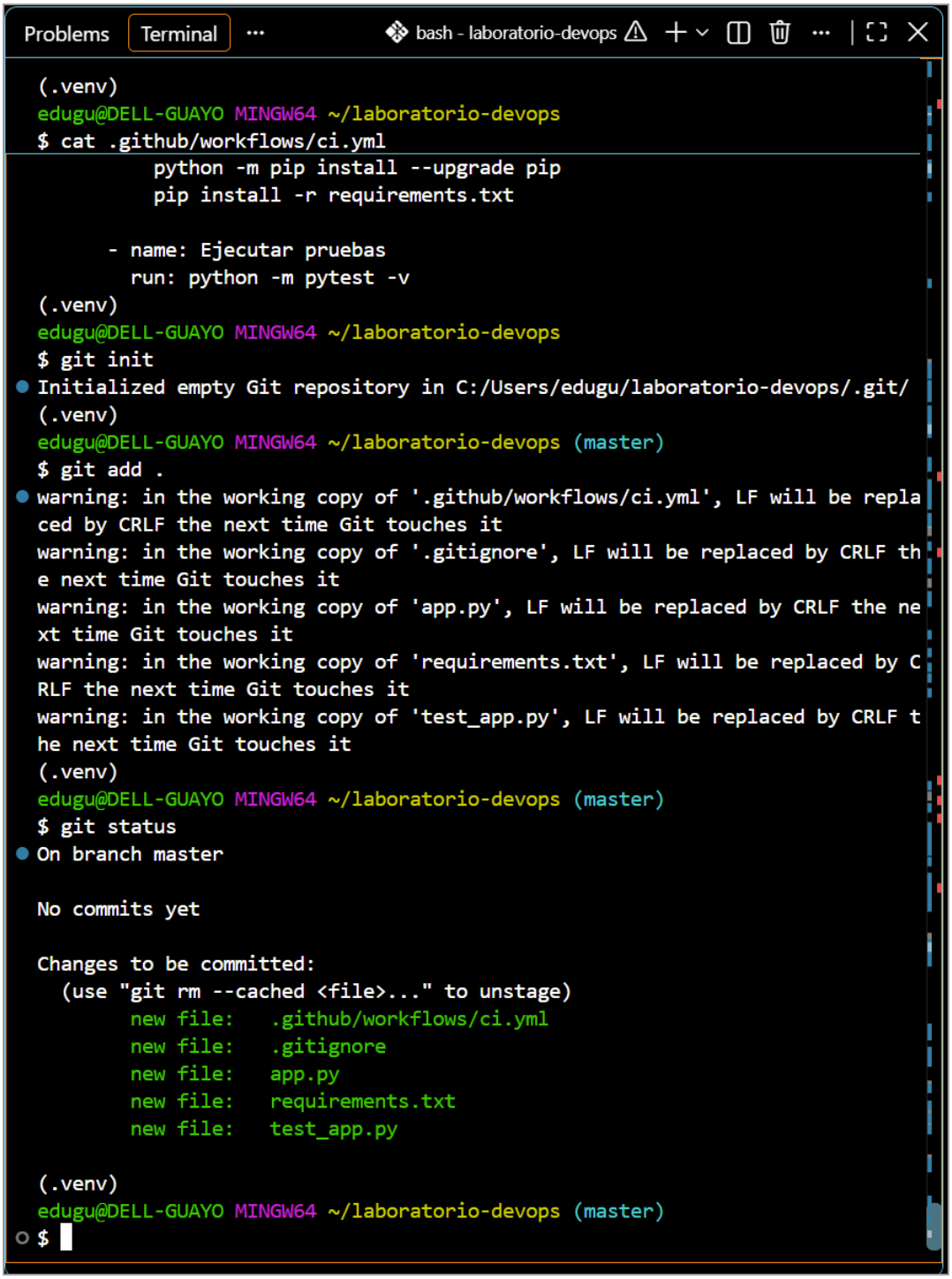


Figura 5. Inicialización del repositorio. Solo se incluyen los 5 archivos del proyecto.

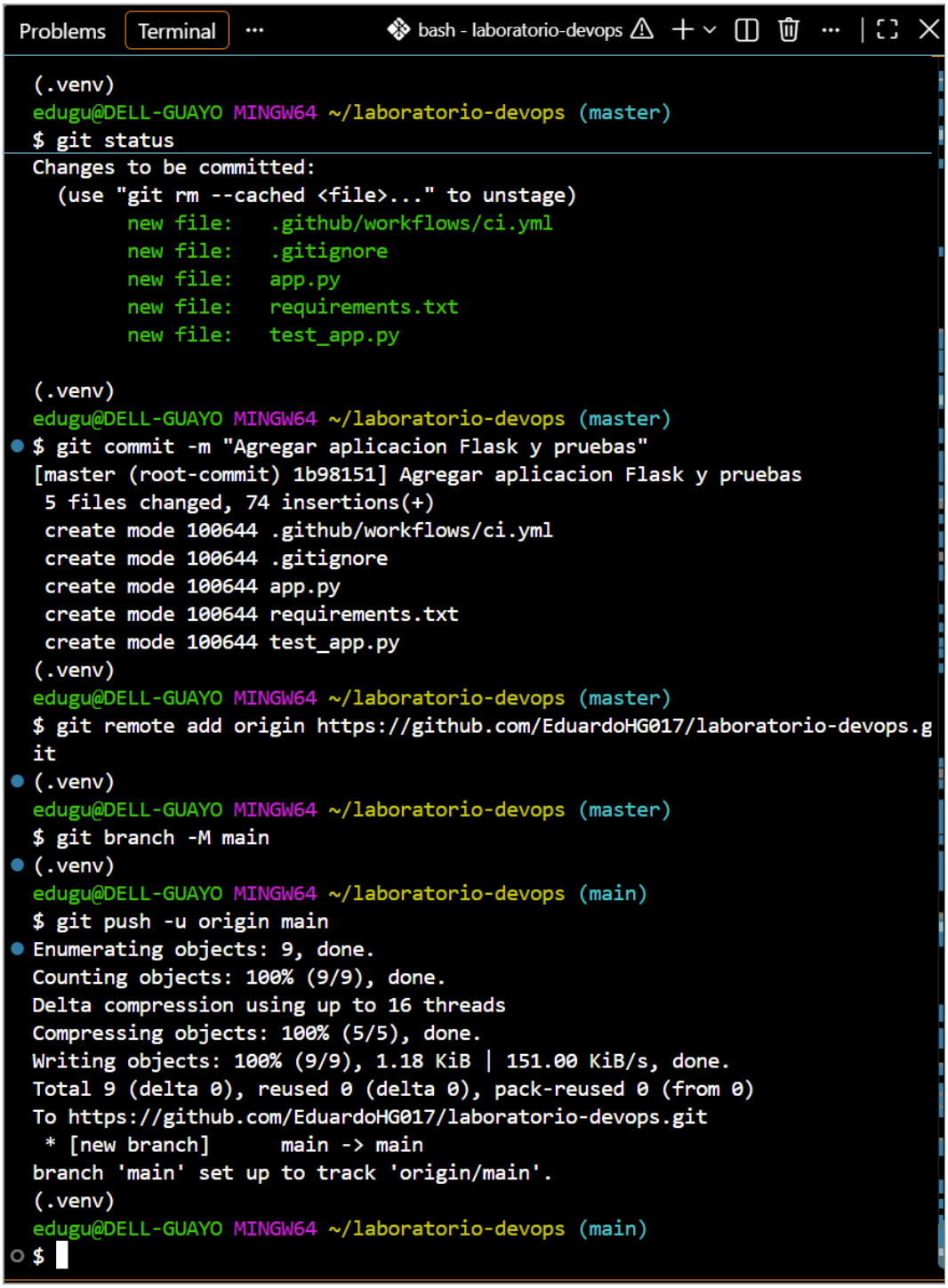


Figura 6. Primer commit y push a GitHub.

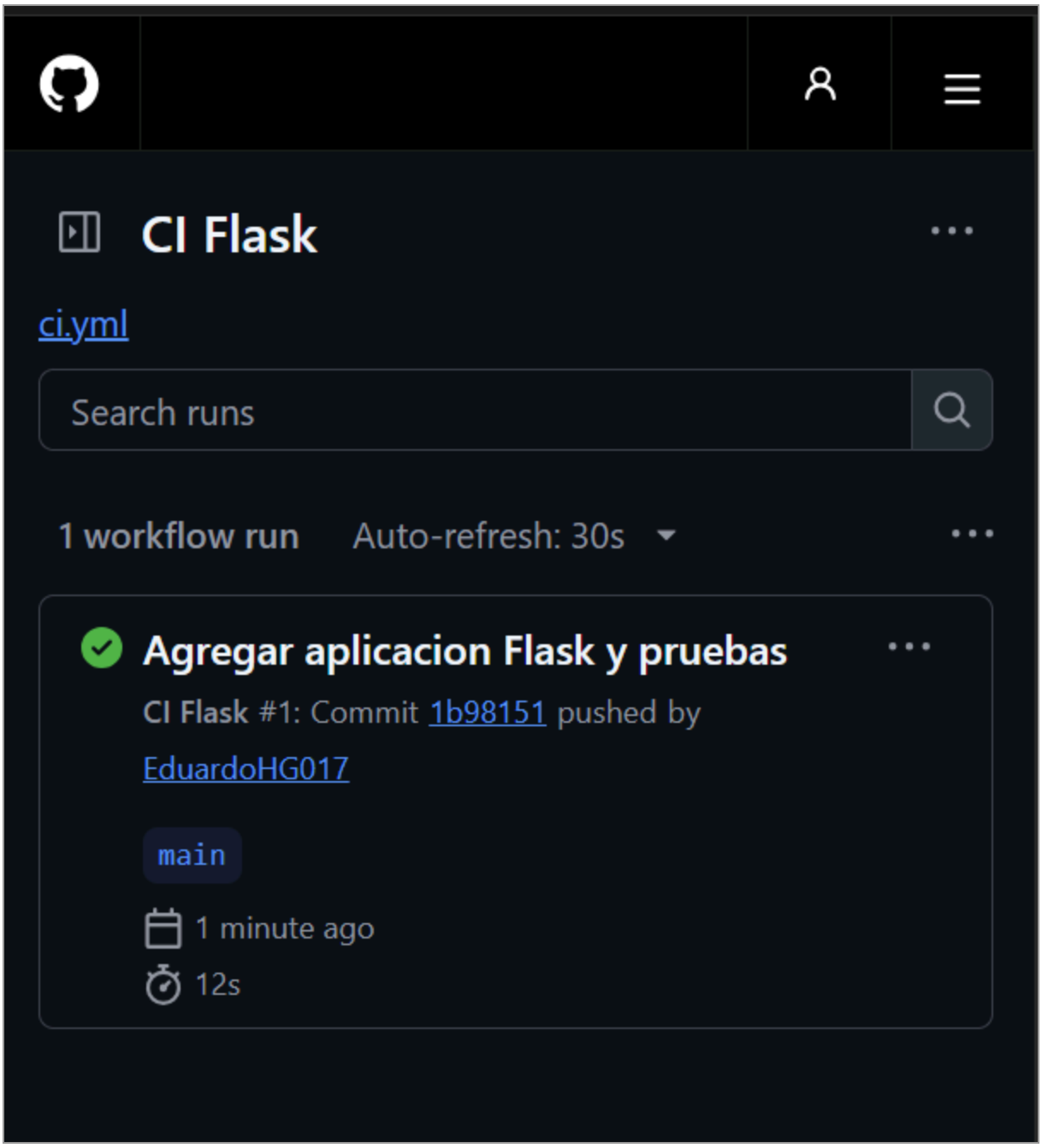


Figura 7. Ejecución #1 de CI Flask, satisfactoria en 12 s.

Parte B. Validación: provocar y corregir un error

A propósito se cambió el mensaje de `/` de "Virtualizacion URL" a "Aplicacion modificada" y se subió en el commit `c3b307d`. La ejecución #2 falló porque `test_inicio` todavía espera el mensaje original. `test_health` siguió pasando, porque ese endpoint no se tocó.

Después se restauró el mensaje original en el commit `7a5134f`, y la ejecución #3 volvió a pasar.

Ejecución	Commit	Mensaje	Resultado
#1	1b98151	Agregar aplicacion Flask y pruebas	Satisfactoria (12 s)
#2	c3b307d	Modificar mensaje de inicio	Fallida (10 s)
#3	7a5134f	Corregir mensaje de inicio	Satisfactoria (18 s)

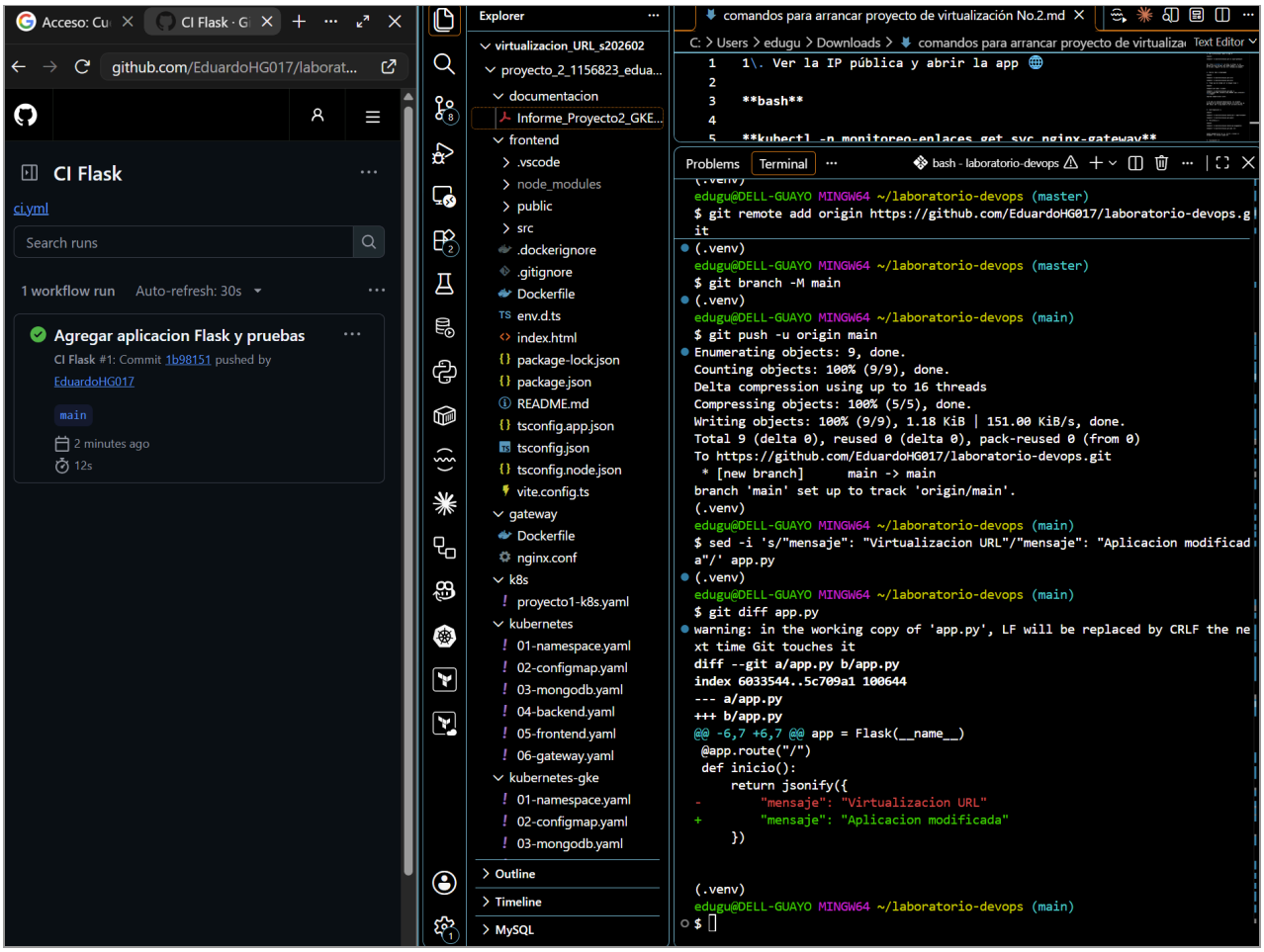


Figura 8. Cambio intencional del mensaje (git diff).

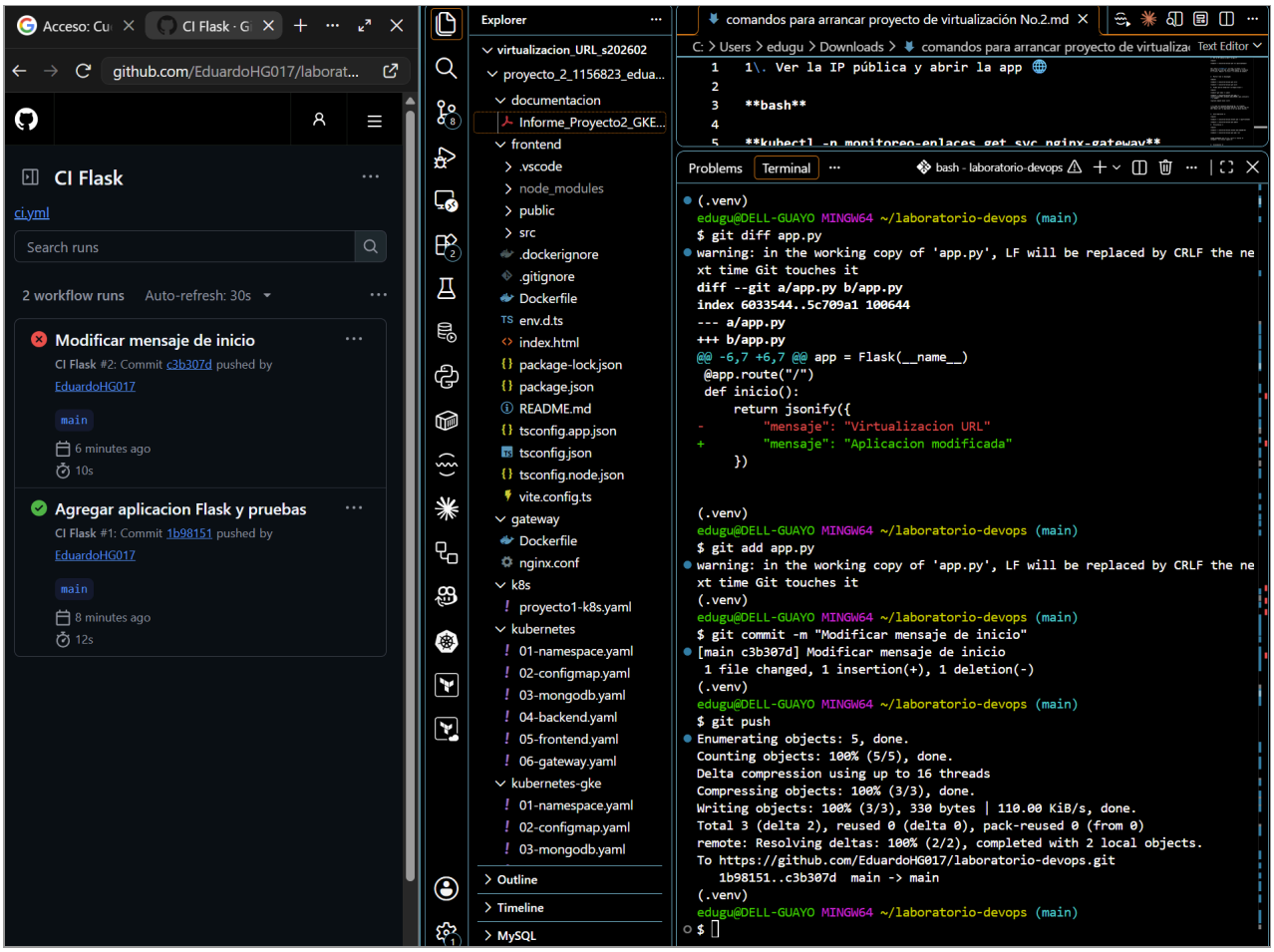


Figura 9. Commit y push del error, y la ejecución #2 fallida en GitHub Actions.

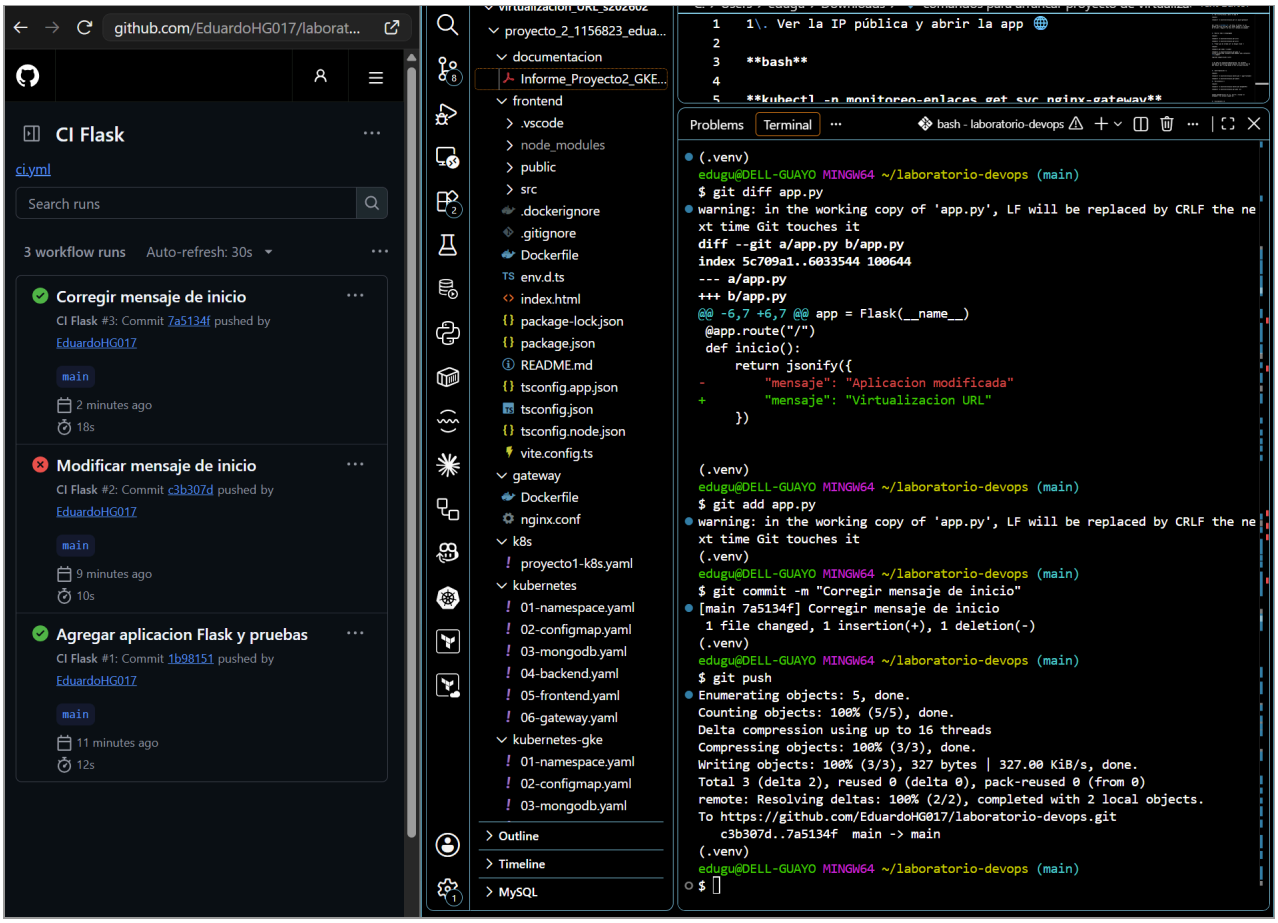


Figura 10. Corrección del mensaje y las tres ejecuciones: satisfactoria, fallida y satisfactoria.

Conclusiones

Con GitHub Actions las pruebas corren solas en cada push, sin tener que ejecutarlas a mano. El error que se introdujo a propósito apareció en unos segundos, y el detalle indicaba qué prueba falló y por qué. Así, un cambio que rompe la aplicación se detecta antes de llegar a producción. Además, el historial de ejecuciones deja registro de qué commit rompió algo y cuál lo corrigió.